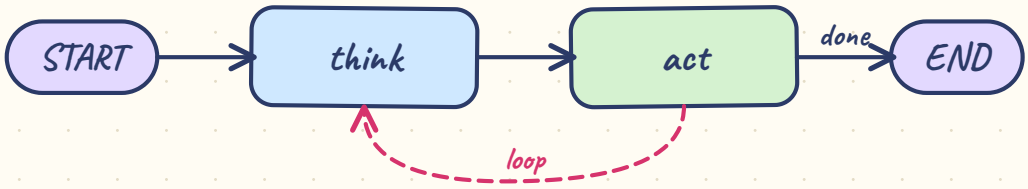
hand-written study notes + engineering handbook

edition 2026 verified

LangGraph Mastery

Agents, graphs, state, reducers, messages, nodes, edges, context, Send, Command and retries: explained from first principles, with diagrams, trade-offs, debugging playbooks and interview answers.

12 lessons | 87+ interview Q&A | code verified by running | 2026 updates



Built from: 5 transcript parts · 12 code files (02–12) · workflows.md. Tested on `langgraph 1.2.16`, `langchain 1.4.6`, Python 3.12.

Part I · Foundations from the lectures (Lessons 1–5) / Part II · Code deep-dives (Lessons 6–12) / Appendices A–F

Contents

Part I — Foundations (from the lecture transcripts)

1 **Agents in LangGraph** — What an agent is, the loop, memory, HITL, streaming — 4
2 **The Graph API** — State, nodes, edges, compile, run, draw — 9
3 **Working with Agent State** — dict · TypedDict · dataclass · Pydantic — 13
4 **State Reducers** — Overwrite, accumulate, merge — 16
5 **The add_messages Reducer** — Chat history, message order, memory — 19

Part II — Code deep-dives (from the repository)

6 **Graph Messages** — MessagesState and simulated conversations — 22
7 **Nodes** — Signatures, config, runtime, per-node policies — 25
8 **Edges** — Fixed, conditional, joins, loops — 28
9 **The Runtime Context** — Typed per-run dependencies — 31
10 **The Send() API** — Map-reduce and dynamic parallelism — 34
11 **Command (bonus)** — Update + route in one return — 37
12 **Retry Policies** — Backoff, jitter, idempotency, resilience — 40

Appendices

A **Series Concept Map** — 43
B **Workflow & Agent Patterns** — 44
C **Debugging Playbook** — 45
D **Source Audit & 2026 Update Notes** — 47
E **One-Page Cheat Sheet** — 49
F **Mock Interview Round** — 50

START HERE

How to use these notes

One consistent shape for every lesson, so you always know where to look.

Every lesson has the same flow

1. **One line** summary and the **What / Why / When / How** card.
2. A **diagram**, then the idea explained in short steps and tables.
3. **Code** from the repo with real, verified output.
4. **Engineering lens**: rules, trade-offs, debugging, 2026 updates.
5. **Interview Q&A** with answers, then **key takeaways**.

Reading the colour boxes

Definition exact meaning of a term

Engineering insight how pros think

Decision rule if this, then that

Watch out / Debug bugs, many reproduced

2026 update what is current in 1.x

Interview tip how to phrase answers

Warm-up recall · Core explain and apply · Deep dive trade-offs and design.

A 7-day study plan

Day	Do this	Goal
1	Lessons 1–2. Build the hello-world graph from memory	Mental model
2	Lessons 3–4. Break and fix a reducer on purpose	State fluency
3	Lessons 5–6. Build a chat graph with a checkpointer	Messages
4	Lessons 7–8. Write a router and unit-test every branch	Control flow
5	Lessons 9–10. Add context and a Send fan-out	Runtime + parallel
6	Lessons 11–12 and Appendix C. Reproduce the bugs	Debugging
7	Appendices E–F. Cover the answers and quiz yourself	Interview ready

LESSON 01

1 Agents in LangGraph

What an agent really is, how its loop works, and what LangGraph gives you.

Source Transcript part 1 · Also workflows.md (workflow vs agent)

In one line: a LangGraph agent is an LLM running in a loop — it thinks, calls tools, reads the results, and repeats until the goal is met.

WHAT

An autonomous unit that takes a task, ***reasons*** through it, takes ***actions*** (tool calls), and stops with a final result.

WHY

LangChain was built for *workflows*. Agents became the way to deliver value with LLMs, so the team made LangGraph, a framework specialised for agents.

WHEN

When the path is ***not known in advance***: open-ended tasks, variable tool use, multi-step research, support, coding.

HOW

Give it a ***prompt***, an ***LLM*** and ***tools***. Let it loop. Add memory, human approval and streaming as needed.

The loop, drawn out

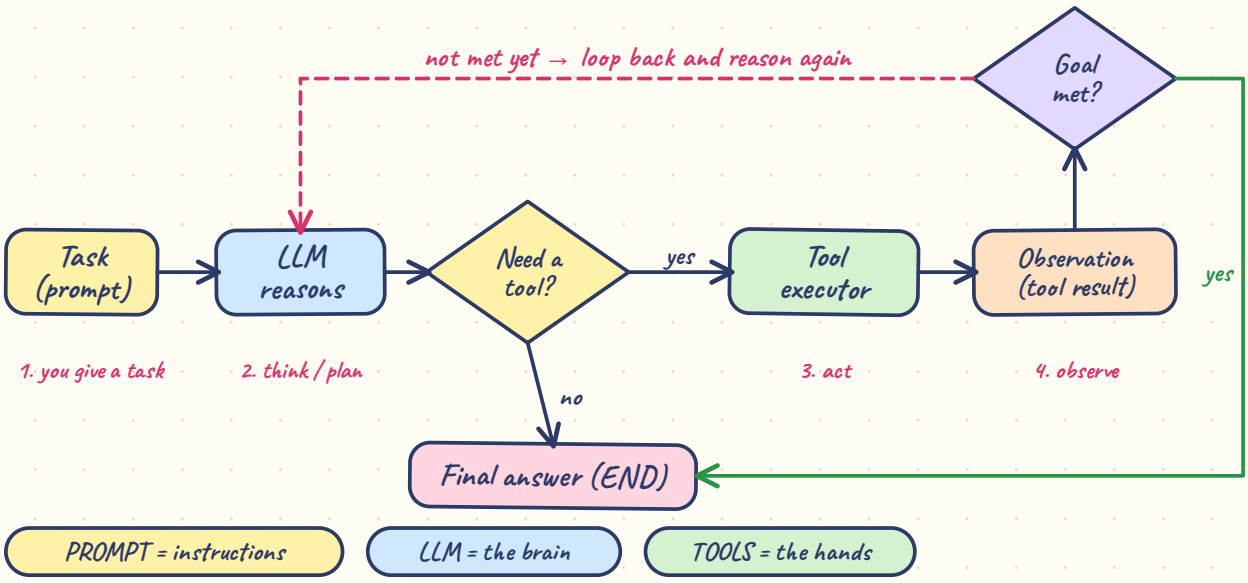


Fig 1.1 — The agent loop (Thought → Action → Observation, repeat)

The three parts of every agent

Part	Job	Analogy	Example
Prompt	Tells the agent *what* to do and *how* to behave	A manager's instructions	"You are a support agent. Never refund over $50."
LLM	Reasons: breaks the task down, picks the next step, decides on tools	The brain	GPT-4o, Claude, Gemini
Tools	Do things in the real world: read, write, call APIs	The hands	send_email(), search_db(), run_sql()

Definition

Agent (LangGraph view) = LLM + tools + prompt, run inside a loop that ends when a *completion requirement* is met. Most people say "LLM + tools". The prompt is the third, quiet part.

Thought → Action → Observation (the ReAct cycle)

1. **Task in.** "Send an email to Ada about Friday's demo."
2. **Reason.** The LLM breaks it down: I need the address → write the body → review it → send.
3. **Decide.** "Do I need a tool?" Real-world effects and data reads usually need one.
4. **Act.** The tool call goes to the **tool executor**, which runs `send_email()`.
5. **Observe.** The result comes back. This result is called an observation.
6. **Check completion.** Done? Stop. Not done? Feed the observation back into reasoning and loop.

Engineering insight

Every loop needs an **exit condition**. "LLM said no more tool calls" is the usual one. Add a hard cap too (recursion/step limit) so a confused model cannot burn money forever. You will meet `recursion_limit` in Lesson 8.

Workflow or agent? Choose on purpose

From Anthropic's *Building Effective Agents* (quoted in the repo's `workflows.md`):

	Workflow	Agent
Who decides the path?	**Your code** (predefined paths)	**The LLM** (dynamic, self-directed)
Predictability	High. Same steps each run	Lower. Steps vary per run
Cost / latency	Low and bounded	Higher and open-ended
Debugging	Easy: read the fixed path	Harder: read the trace
Best for	Known procedures (extract → validate → save)	Open problems (research, triage, coding)

Decision rule

Start with the **simplest thing that works**: a single LLM call → a workflow → only then an agent. Add autonomy when the task path truly cannot be written down. Autonomy costs predictability.

Beyond the basics: what LangGraph adds

Feature	What it means	Why you care
Short-term memory	State of the ***current run***. Gone when the run ends (like RAM)	Keeps context while working
Long-term memory	Data stored ***outside*** the run: user base, facts, preferences (like a hard drive)	Survives restarts; too big for the context window
Human in the loop	Agent ***pauses*** so a person can approve, edit, or take over	Safety for emails, payments, deletes
Streaming	Live tokens, steps, tool results, reasoning	UX, logging, cost tracking, debugging

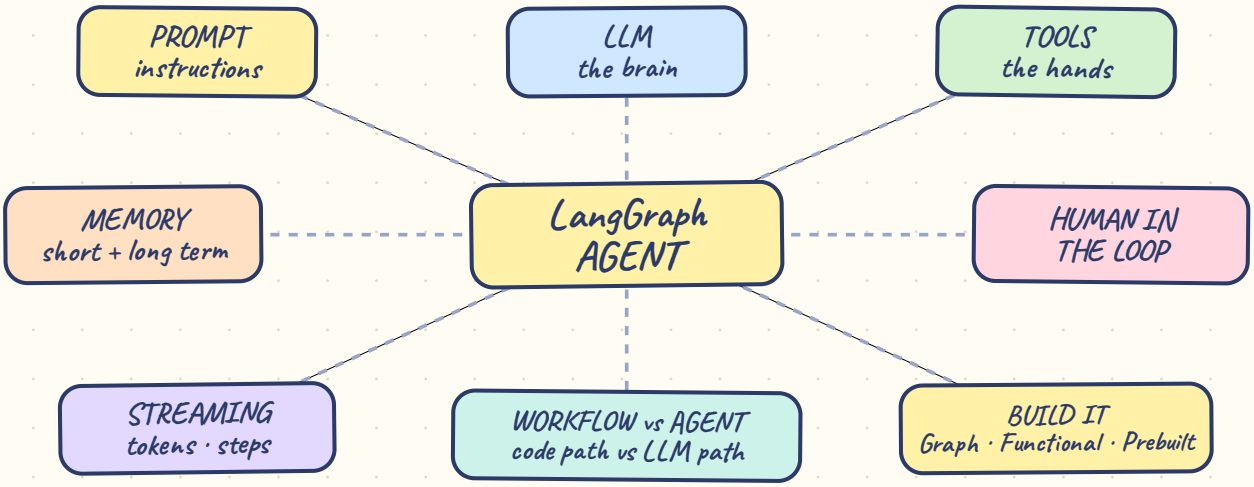


Fig 1.2 — Concept map for Lesson 1

Two ways to build in LangGraph

Path	What you get	Pick it when
Primitives: Graph API	Nodes, edges, state as a graph. Most popular. The name "LangGraph" comes from it	You want full control and visible structure
Primitives: Functional API	Normal imperative Python (`@entrypoint` , `@task`)	You prefer plain code over graph wiring
Prebuilt components	A ready-made ReAct-style agent with memory, HITL, streaming wired in	You want speed, and defaults are fine

← Course plan: learn primitives first, then you can judge and customise the prebuilt ones.

2026 update

- ***LangGraph 1.x is stable*** (1.0 shipped Oct 2025). This book was verified on `langgraph 1.2.16`, `langchain 1.4.6`, Python 3.12.
- The fast path for a prebuilt agent is now `from langchain.agents import create_agent` (verified: it exists and returns a compiled LangGraph graph). The older `langgraph.prebuilt.create_react_agent` still imports, but prefer `create_agent` for new code. Check the docs for the current deprecation status.
- `create_agent` accepts `system_prompt`, `middleware`, `checkpointer`, `store`, `context_schema`, and `response_format`. It is the prompt + LLM + tools + loop you just learned.

python (illustrative, needs an API key) — *prebuilt agent*

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b

agent = create_agent(
    model="openai:gpt-4o",          # provider:model string
    tools=[multiply],               # the hands
    system_prompt="You are a careful math helper.",  # the prompt
)
result = agent.invoke({"messages": [("user", "What is 6 times 7?")]})
print(result["messages"][-1].content)
```

Interview preparation

Q1. What is an agent in LangGraph?

Warm-up

An ***autonomous unit*** made of a ***prompt, an LLM and tools***, running in a ***loop***. The LLM reasons about a task, decides whether to call a tool, reads the observation, and repeats until a completion condition is met.

Q2. Why is the LLM called the "brain" and tools the "hands"?

Warm-up

The LLM only produces text (including *requests* to call tools). It cannot touch the world. Tools do the real work (APIs, files, databases). The agent loop connects both.

Q3. Explain the Thought → Action → Observation cycle. Where does the loop stop?

Core

- ***Thought:*** LLM breaks down the task, picks a step.
- ***Action:*** it emits a tool call; an executor runs it.
- ***Observation:*** the tool result returns to the LLM.

It stops when the model answers ***without*** a tool call (completion requirement met) *or* a safety cap such as a recursion limit is hit.

Q4. Workflow vs agent: how do you decide which to build?

Core

Ask: ***can I write the steps down in advance?*** Yes → workflow (cheaper, predictable, testable). No → agent. Start simple. Many production systems are ***workflows with one agentic step*** inside.

Trade-off: agents give flexibility but cost more tokens, add latency, and are harder to evaluate.

Q5. Short-term vs long-term memory in an agent? Core

Short-term = state of the current run/thread (RAM). **Long-term** = persisted store across sessions, such as user profiles or semantic facts (hard drive). Use long-term when data is large, shared, or must outlive the run. In LangGraph: a **checkpointer** gives thread memory; a **store** gives cross-thread memory.

Q6. Your agent works in demos but loops forever in production. How do you debug and harden it? Deep dive

- **Observe first:** stream steps or trace the run. Read the actual tool calls and observations.
- **Likely causes:** vague tool descriptions, tool errors the model keeps retrying, no clear "done" criterion in the prompt.
- **Guardrails:** set a recursion limit; return *clear* tool errors; add retries only for transient failures; add human approval for risky tools.
- **Design fix:** if the path is known, turn it into a workflow with fixed edges.

Q7. When would you use primitives instead of a prebuilt agent? Deep dive

When you need **custom control flow** (branching, parallel fan-out, custom state, special approval steps) or must **explain and test every step**. Prebuilt = fast start, less control. Primitives = more code, full control. Knowing primitives also lets you customise prebuilt agents safely.

Interview tip

Say the one-liner: *"An agent is an LLM in a loop with tools, and the engineering is in the loop's state, exit conditions and guardrails."*

Key takeaways

- Agent = **prompt + LLM + tools** in a **loop**.
- The loop is **Thought** → **Action** → **Observation**, repeated until completion.
- Prefer workflows when the path is known. Use agents when it is not.
- Memory (short/long), human-in-the-loop and streaming come built in.
- Build with **primitives** for control, or **prebuilt** (`create_agent`) for speed.

LESSON 02

2 The Graph API

Build your first graph: state, nodes, edges, compile, run, draw.

Source Transcript part 2 · Code 02-graph-api.py

In one line: the Graph API models an agent as **state + nodes + edges**, compiles it into a runnable, and runs it like a state machine.

WHAT

One of two ***primitive*** ways to build agents. It represents decisions, actions and flow as a ***graph***.

WHY

Explicit structure: you can ***see, test, pause, resume and debug*** each step instead of hiding logic inside one big loop.

WHEN

Any multi-step LLM app: chat bots, tool agents, pipelines, branching or parallel work.

HOW

Define state → add nodes → connect edges → ***compile*** → invoke.

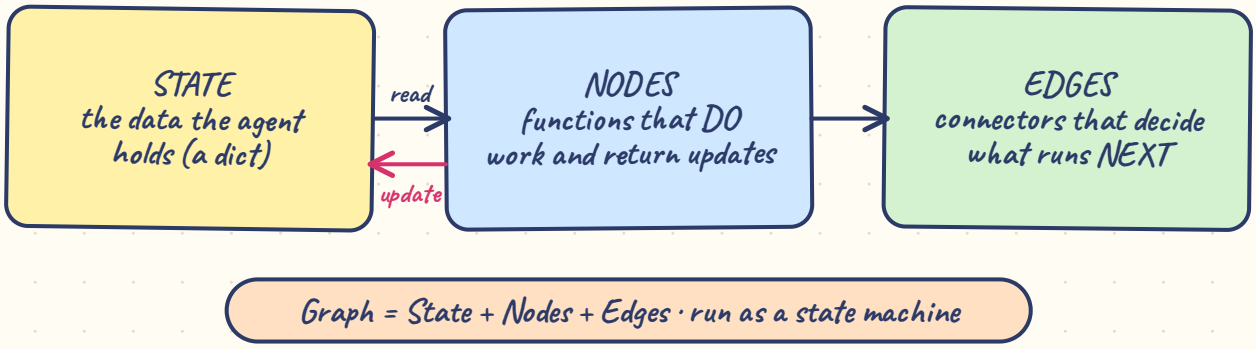


Fig 2.1 — The three building blocks

State, nodes, edges

Part	What it is	Key rules
State	Shared data the agent needs to do its job. Usually a dictionary of ***state variables***	Define it ***first***. Nodes read it and return *partial* updates
Node	A normal ***Python function*** (or JS function). A step in the agent: call an LLM, validate, draft, send	Receives state. Returns only the keys it wants to change
Edge	A connector between nodes. Decides the next step	***Fixed*** (A → B) or ***conditional*** (a function picks)
START / END	Two ***built-in*** special nodes: where the graph begins and stops	Import from `langgraph.graph`

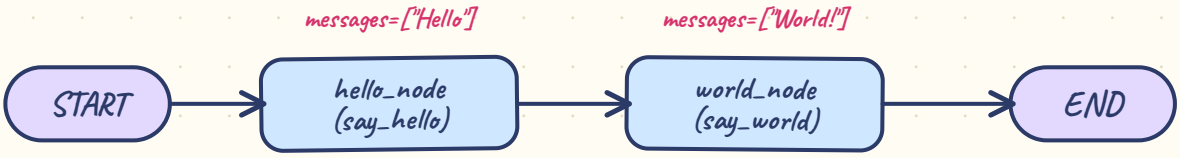
The 5 steps to build any graph

1. **Define the state** (a `TypedDict` works well).
2. **Create the graph**: `StateGraph(State)`.
3. **Add nodes**: `graph.add_node("name", func)`.
4. **Connect edges**: `START → … → END` with `add_edge`.
5. **Compile**, then run: `graph.compile()` → `.invoke(initial_state)`.

Watch out / Debug

`add_edge` uses the **node names** you gave in `add_node`, not the function names. Mixing them up is the classic first-hour error.

Hello-world graph (from the repo)



state: [] → [Hello] → [Hello, World!] (add_messages appends)

Fig 2.2 — The 02-graph-api.py graph

02-graph-api.py — trimmed

```python
class SimpleState(TypedDict):
    messages: Annotated[list, add_messages]   # reducer = append

def say_hello(state: SimpleState):
    return {"messages": ["Hello"]}            # partial update only
def say_world(state: SimpleState):
    return {"messages": [" World!"]}

graph = StateGraph(SimpleState)
graph.add_node("hello_node", say_hello)
graph.add_node("world_node", say_world)
graph.add_edge(START, "hello_node")
graph.add_edge("hello_node", "world_node")
graph.add_edge("world_node", END)

app = graph.compile()                         # ← do not forget
final = app.invoke({"messages": []})
```

```
Executing 'say_hello` node...
Executing 'say_world` node...
['Hello', ' World!']   ← message.content of two HumanMessage objects
```

← Plain strings became HumanMessage objects: add_messages converts them (Lesson 5).

What compile() really gives you

- A ***runnable***: `invoke`, `batch`, `stream`, and async versions.
- Options at compile time: `checkpointer` (memory), `interrupt_before/after`, `store`.
- Config at run time: `thread_id`, `recursion_limit`, plus run-scoped `context`.

Engineering lens

Engineering insight

Supersteps. LangGraph runs in discrete steps (a message-passing model inspired by Google's Pregel). In each step, every node that is ready runs, their updates are applied through the ***reducers***, then the next step begins. This is why parallel branches, checkpoints and replays are possible.

Decision rule

- Put ***one responsibility*** in each node. Small nodes are easy to test, retry and trace.
- Keep nodes ***pure-ish***: read state, do work, return an update. Hidden global state breaks replay.
- Use ***edges for control flow***, not `if`-ladders inside one giant node.

Debug note (verified)

ASCII drawing fails without `grandalf`. Verified: `get_graph().draw_ascii()` raises `ImportError: Install grandalf` until you `pip install grandalf`. For a picture, use `draw_mermaid_png()` or LangGraph Studio.

2026 update

Prefer `StateGraph(State, context_schema=…, input_schema=…, output_schema=…)` (verified signature). Python 3.10+ is required by modern LangGraph. The ***Functional API*** (`@entrypoint`, `@task`) is the other primitive: same runtime, written as plain code.

Interview preparation

Q1. What are the three core parts of the Graph API? Warm-up

State (shared data), ***nodes*** (functions doing work and returning updates), ***edges*** (connectors defining flow). Plus the built-in `START` and `END` nodes.

Q2. Why must you call `compile()`? Warm-up

`StateGraph` is a *builder*. `compile()` validates the structure (e.g. unknown nodes, unreachable parts) and returns an executable ***runnable*** that supports `invoke`, `batch`, `stream` and takes checkpointers, etc.

Q3. Does a node have to return the whole state? Core

No. Return ***only the keys you change***. LangGraph merges them into state through each key's reducer. Returning the full state is wasteful and, with reducers such as append, can cause duplicates.

Q4. A conditional edge sends you to a node that does not exist. What happens and how do you prevent it?

Core

You get an error at compile time (if you declare the path map) or at run time (a routing key that is not mapped). Prevent it by typing the router as `Literal[...]` or giving an explicit path map, and testing every branch. See Lesson 8.

Q5. How does LangGraph execute parallel branches, and why does that affect state design?

Deep dive

It uses ***supersteps***: all ready nodes run in the same step, then their writes are applied. If two nodes write the same key in one step, the key ***needs a reducer*** or you get `InvalidUpdateError` (verified in Lesson 4). So design state keys with concurrency in mind.

Q6. Graph API vs Functional API: how do you choose?

Deep dive

Both share the same runtime (persistence, streaming, HITL). Graph API is ***declarative and visual***, good for complex branching and team review. Functional API is ***imperative***, good when the logic reads naturally as Python with loops and conditionals. Pick by readability and how much explicit structure you want in traces and diagrams.

Key takeaways

- Graph = ***State + Nodes + Edges***, executed as a state machine in supersteps.
- Five steps: ***state*** → ***graph*** → ***nodes*** → ***edges*** → ***compile***, then `invoke`.
- Nodes return ***partial updates***. Edges use ***node names***.
- `START` and `END` are built in. Install `grandalf` to draw ASCII.

LESSON 03

3 Working with Agent State

Schemas: dict, TypedDict, dataclass and Pydantic, and when to use each.

Source Transcript part 3 · Code 03-graph-state.py

In one line: state is the data your agent carries between nodes. It has a **schema** (its shape) and **reducers** (how each key updates).

WHAT

The ***pieces of data*** the agent needs during one run. Not fetched from a persistent source; it is the working set.

WHY

Everything else (nodes, edges) reads and writes it. A clean schema = clean agent. "Your graph is a state engine."

WHEN

Decide it ***first***, before any node. Revisit when a node needs data nobody holds yet.

HOW

Pick a schema style (dict, TypedDict, dataclass, Pydantic), add reducers with `Annotated`.

Definition

State = schema + reducers. The ***schema*** is the data model (keys and types). A ***reducer*** decides how an incoming update merges into one key. Every key has its own reducer (default: overwrite).

Four ways to write a schema

Style	Defaults?	Validation?	Speed	Use when
Plain `dict`	No	No	Fastest	Throwaway demos. Only default reducer. Weak typing
`TypedDict`	No	No (hints only)	Fast	***Recommended default.*** Simple, works with Annotated reducers
`@dataclass`	Yes	No	Fast	You want default values and dot access
Pydantic `BaseModel`	Yes	***Yes***	Slower	You need runtime validation, constraints, rich types

← *Instructor's note: TypedDict and dataclass are faster than Pydantic. Pydantic gives more options.*

Example: a class-trip eligibility agent

- `total_students: int`, `average_score: float`, `qualified_students: list[str]`.
- Nodes will read the first two and fill the third. Initial values can be given for some keys.

Same nodes, three schemas

03-graph-state.py — schemas

```python
def custom_add(current: int, new: int):          # a reducer
    return current + new

class TypedDictState(TypedDict):
    messages: Annotated[List[str], add_messages]
    step_count: Annotated[int, custom_add]
    private_data: str

class PydanticState(BaseModel):
    messages: Annotated[List[BaseMessage], add_messages] = Field(default_factory=list)
    step_count: Annotated[int, custom_add] = Field(default=0)
    private_data: str = Field(default="")
```

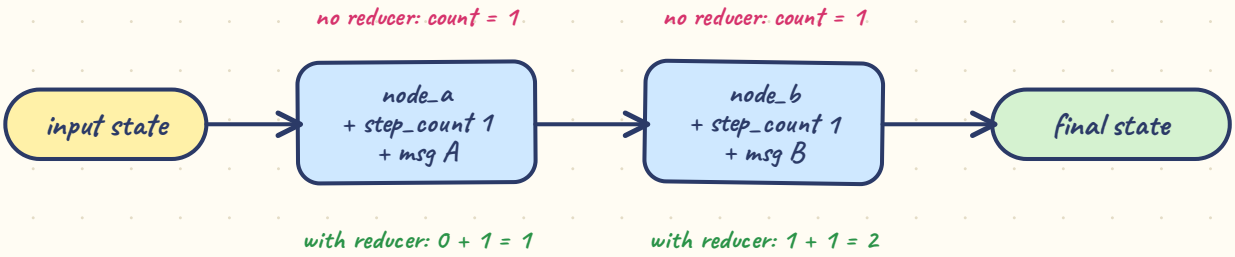


Fig 3.1 — Same two nodes. Without a reducer count stays 1; with `custom_add` it reaches 2

terminal — verified output

```
# METHOD 1: plain dict  (default reducer = overwrite)
Final State: {'messages': ['Step B Completed'], 'step_count': 1}

# METHOD 2/4: TypedDict or Pydantic  (reducers on step_count + messages)
step_count: 2     messages: [HumanMessage('Step A…'), HumanMessage('Step B…')]
```

```
dict           → messages overwritten, step_count = 1
Pydantic/dataclass → messages appended, step_count = 2
(both returned a plain dict from invoke)
```

Engineering lens

1. **List what the agent must remember** to finish the task. Nothing more.
2. **Choose the schema style** using the table above.
3. **For each key ask:** overwrite, append, or merge? That picks the reducer.
4. **Separate private from public**: use `input_schema` / `output_schema` so callers see only what they should.

Engineering insight

- **Pydantic mutable default trap:** use `Field(default_factory=list)`, never `= []`. It gives every instance a *fresh* list.
- **Keep state small.** Big blobs are copied into every checkpoint. Store ids or references, not megabytes.
- **Name keys for meaning**, not for the node that writes them.

2026 update

- Verified on 1.2.16: `StateGraph(State, input_schema=In, output_schema=Out)` returns only the output keys (test returned `{'ans': '42'}` and hid the scratch key).
- Both Pydantic and dataclass states returned ***dicts*** from `invoke` in my run. Do not assume you get your model back.
- Pydantic validation is meant for data *entering* the graph. Do not rely on it to police every node's output.

Interview preparation

Q1. What is state in a LangGraph agent? Warm-up

The data held during a run. It has a ***schema*** (keys and types) and ***reducers*** (merge rules). Nodes read it and return updates.

Q2. Name the ways to define a state schema. Warm-up

Plain dict, `TypedDict`, `dataclass`, Pydantic `BaseModel`. TypedDict is the common default; Pydantic when you need validation.

Q3. Why did a plain-dict state end with `step_count = 1` after two nodes each added 1? Core

No reducer was set, so the ***default reducer overwrites***. Node A wrote 1, node B wrote 1; last write wins. Add `Annotated[int, custom_add]` to get 2.

Q4. TypedDict vs dataclass vs Pydantic: trade-offs? Core

- ***TypedDict:*** light, fast, no defaults/validation.
- ***dataclass:*** defaults, still light.
- ***Pydantic:*** validation and constraints, but more overhead and more surprising behaviour.

Start with TypedDict. Move to Pydantic only when bad data is a real risk.

Q5. How do you stop internal scratch keys leaking to API callers? Deep dive

Use separate ***input*** and ***output*** schemas. The full state holds scratch keys; the output schema exposes only the final answer. I verified this: the scratch key was not returned.

Q6. What goes wrong when state gets too big? Deep dive

Every checkpoint stores the state, so memory, storage and latency grow. Prompts also bloat if you pass everything to the LLM. Keep large documents in an external store and keep ids in state; trim message history.

Key takeaways

- State = ***schema + reducers***. Define it first.
- ***TypedDict*** is the practical default; Pydantic adds validation at a cost.
- Default reducer ***overwrites***. Add reducers for counters and lists.
- Use `default_factory` for mutable defaults and I/O schemas to hide internals.

LESSON 04

4 State Reducers

How state keys change: overwrite, accumulate, merge.

Source Transcript part 4 · Code 04-reducers.py

In one line: a reducer is a function `(current, new) → merged` that controls how one state key changes every time a node returns an update.

WHAT

A ***pure function*** attached to a state key. LangGraph calls it whenever a node writes that key.

WHY

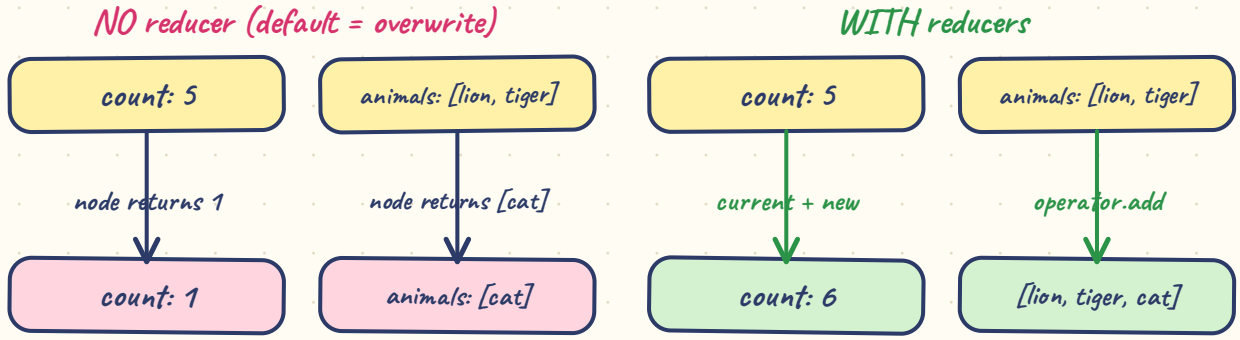
Without it, every write ***replaces*** the old value. Counters, lists and chat history need merging, not replacing.

WHEN

Any key written more than once, written by parallel nodes, or accumulated across steps.

HOW

Write `def r(current, new)`, attach with `Annotated[type, r]`. Or reuse `operator.add`, `add_messages`.



the reducer receives (current value, new value) and returns the merged value

Fig 4.1 — Overwrite vs reduce (values from 04-reducers.py)

Default vs custom reducer

	Default reducer	Custom reducer
Rule	New value ***replaces*** old	Your function decides
Setup	Nothing. Just a plain type	`Annotated[int, my_func]`
Good for	Status flags, latest answer, current user input	Counters, lists, merged dicts, chat history
Risk	Silent data loss (earlier values vanish)	Wrong merge logic, non-deterministic with parallel writes

How to write and attach one

04-reducers.py — trimmed

```python
from operator import add

def custom_increment(current: int, new: int) -> int:
    return current + new                    # your merge rule

class StateWithCustomReducer(TypedDict):
    count:   Annotated[int, custom_increment]  # type first, reducer second
    animals: Annotated[List[str], add]         # list + list = append

def node_to_update(state):
    return {"count": 1, "animals": ["cat"]}
```

```
No reducer : count 5 → 1      animals [lion, tiger] → ['cat']
Reducers   : count 5 → 6      animals [lion, tiger] → ['lion','tiger','cat']
```

How Annotated works

In `Annotated[T, r]`, the ***first*** argument is the type. Anything ***after*** it that is a function is picked up by LangGraph as the reducer. Functions from other libraries work if they follow `(current, new)`.

Reducers and parallel writes (verified)

terminal — verified on langgraph 1.2.16

```
# two nodes both return {"x": ...} in the SAME step, no reducer on x
InvalidUpdateError: At key 'x': Can receive only one value per step.
Use an Annotated key to handle multiple values.
```

Engineering insight

A reducer is also a ***concurrency contract***. With parallel branches (Lesson 10), the order of arrival is not guaranteed. Prefer reducers that are ***associative and order-insensitive*** (sum, set-union, append-then-sort), or sort later.

Decision rule

- ***Replace*** → default. "Latest wins" is correct (status, current answer).
- ***Accumulate numbers*** → `current + new`.
- ***Accumulate lists*** → `operator.add`.
- ***Chat messages*** → `add_messages` (Lesson 5).
- ***Merge dicts*** → a small custom merge function.

Watch out / Debug

- ***"My list only has the last item."*** → You forgot the reducer. Default overwrote.
- ***"My counter doubled."*** → The node returned the *total*, but the reducer adds. Return the *delta* instead.
- ***Duplicates after re-invoke.*** → Appending reducers re-add input. Use ids (`add_messages` dedupes by id).

2026 update

Reducers also run on the ***initial input*** you pass to `invoke`. Keep them ***pure*** (no I/O, no randomness): checkpoint replay and time-travel re-run state merges.

Interview preparation

Q1. What does a reducer do? — Warm-up

It controls how a node's update is merged into a state key: `new_value = reducer(current, update)`.

Q2. What is the default reducer's behaviour? — Warm-up

Overwrite. The new value replaces the whole old value, even for lists. That is why a list ends with only the latest item.

Q3. Write a reducer that merges two dicts and explain where you attach it. — Core

`def merge(cur, new): return {**cur, **new}`, attached as `Annotated[dict, merge]`. Be careful about key collisions: later wins.

Q4. Why does the node return `{'count': 1}` instead of the new total? — Core

With a summing reducer, the node returns the ***delta***. The reducer computes the total. This keeps nodes independent of the current value, which is critical when nodes run in parallel.

Q5. Two parallel nodes write the same key. What happens, and what are your options? — Deep dive

Without a reducer LangGraph raises `InvalidUpdateError` ("Can receive only one value per step"). Options: (1) add a reducer to merge, (2) have each node write a ***different key*** and merge in a later node, (3) serialise the nodes.

Q6. What properties should a reducer have, and why? — Deep dive

- ***Pure***: no side effects, because state merges are replayed from checkpoints.
- ***Deterministic*** and ideally ***order-insensitive*** for parallel writes.
- ***Cheap***: it runs on every write of that key.

Key takeaways

- Reducer signature: `(current, new) → merged`. Attach with `Annotated`.
- Default = ***overwrite***. Lists and counters need explicit reducers.
- Return ***deltas***, let the reducer combine.
- Parallel writes to a key ***require*** a reducer (or you get `InvalidUpdateError`).

LESSON 05

5 The add_messages Reducer

Managing LLM conversations: message types, order, ids, memory.

Source Transcript part 5 (cut off mid-demo) · Code 05-add_messages.py, 04-reducers.py

In one line: `add_messages` is LangGraph's built-in reducer for chat history: it appends, orders, and de-duplicates LLM messages.

WHAT

A ready-made reducer in `langgraph.graph.message` that manages a list of ***LangChain message objects***.

WHY

LLMs are ***stateless***: each call must include the whole conversation. Messages also have strict structure and order.

WHEN

Any conversational agent, tool-calling agent, or human-in-the-loop flow. Whenever a state key holds chat history.

HOW

`messages: Annotated[list[BaseMessage], add_messages]`. Return new messages from nodes. It appends them.

The message types

Type	Who writes it	Purpose
`SystemMessage`	You (developer)	Sets behaviour. ***One*** per conversation, at the top
`HumanMessage`	The user	A prompt or reply
`AIMessage`	The LLM	A response. May carry `tool_calls` (requests for tools)
`ToolMessage`	Your tool runner	The ***result*** of a tool call, sent back to the LLM

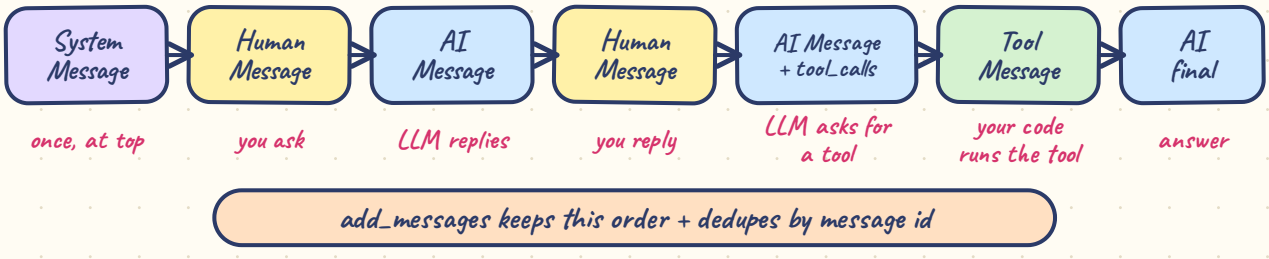


Fig 5.1 — A valid conversation order

Clarification

Correcting a common mix-up: the LLM does *not* send a "tool message". It sends an `AIMessage` ***containing*** `tool_calls`. The `ToolMessage` is what *your code* returns after running the tool. Most providers reject a tool call that is not followed by its matching `ToolMessage`.

What add_messages does (verified)

You return	Result
A single message or a list	Both work. It normalises to a list and **appends**
Plain strings	Converted to `HumanMessage` (that is why Lesson 2 printed message objects)
A message with an existing id	**Replaces** that message (edit in place)
Two AIMessages in a row, new ids	**Not merged.** Both kept
`RemoveMessage(id=…)`	Deletes that message by id

← The lecture says AI messages "merge". In 1.2.16 they did not. The id rule is the real behaviour.

Chat node + two turns (from the repo)

05-add_messages.py — trimmed

```python
llm = ChatOpenAI(model="gpt-4o")

class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

def chat_node(state: AgentState) → dict:
    response = llm.invoke(state["messages"])   # send FULL history
    return {"messages": response}              # reducer appends it

# turn 1
t1 = agent.invoke({"messages": HumanMessage(content="Hi, I am FK")})
# turn 2: no memory unless you pass history back
t2 = agent.invoke({"messages": t1["messages"] + [HumanMessage("Favourite colour?")]})
```

Better in 2026: let a checkpointer remember

python — verified

```python
from langgraph.checkpoint.memory import InMemorySaver
app = graph.compile(checkpointer=InMemorySaver())
cfg = {"configurable": {"thread_id": "t1"}}
app.invoke({"messages": [HumanMessage("a")]}, cfg)
r = app.invoke({"messages": [HumanMessage("b")]}, cfg)   # only the NEW message
```

```
['a', 'seen 1', 'b', 'seen 3']    # thread kept history, bot saw 3 messages on turn 2
```

Engineering insight

- Without a checkpointer each `invoke` starts empty, so re-sending history is ***required***. With one, send only the new message and pass a `thread_id`.
- For production use a database-backed saver (e.g. Postgres) instead of `InMemorySaver`.
- Long chats overflow the context window: ***trim*** (`trim_messages`) or ***summarise*** and remove old messages with `RemoveMessage`.

Decision rule

Chat history in state → ***always*** use `add_messages`. Do not hand-roll with strings. You lose message types, ids, tool-call structure and multimodal content parts.

2026 update

The demo model name (`gpt-4o`) will date quickly. Swap in any chat model. In LangChain 1.x you can also write `init_chat_model("provider:model")` . The transcript ends while writing this demo's imports; the logic above comes from the repo file.

Interview preparation

Q1. Why do you pass the whole history to the LLM every turn? — Warm-up

LLMs are ***stateless***. Context only exists in the prompt you send. The state (or a checkpointer) stores it for you.

Q2. What is the correct sequence for a tool call? — Core

Human → `AIMessage(tool_calls=[…])` → your code runs the tool → `ToolMessage(tool_call_id=…)` → LLM again → final `AIMessage` .

Q3. How would you edit or delete a message already in state? — Core

Return a message with the ***same id*** to replace it, or `RemoveMessage(id=…)` to delete it. That works because `add_messages` merges by id.

Q4. Conversation history grows past the context window. What do you do? — Deep dive

- ***Trim:*** keep the last N tokens/messages (always keep the system message).
- ***Summarise:*** replace old messages with a short summary node + `RemoveMessage` .
- ***Offload:*** store facts in long-term memory and retrieve on demand.
- ***Never*** cut between an AI tool-call and its ToolMessage.

Q5. Your API returns 400: "tool_calls must be followed by tool messages". Where do you look? — Deep dive

History is malformed: a trim/removal split an AI tool-call from its ToolMessage, or a tool node failed to return one. Print the message list with types and ids. Fix trimming to remove them ***as a pair***, and always return a ToolMessage (even for errors).

Key takeaways

- `add_messages` = the reducer for chat history. Appends, normalises, de-dupes by id.
- Order matters: System → Human ↔ AI, with ***AI(tool_calls)*** → ***ToolMessage*** pairs.
- Without a checkpointer, re-send history. With one, use `thread_id` .
- Plan for long chats: trim, summarise, or offload.

LESSON 06

6 Graph Messages

Using MessagesState and simulating a conversation inside a graph.

Code 06-graph-messages.py (no transcript provided; explained from the code and verified by running it)

In one line: MessagesState is a prebuilt state that already has a `messages` key with `add_messages`. Subclass it and add your own keys.

WHAT

A ready-made `TypedDict` with `messages` + the `add_messages` reducer already attached.

WHY

Saves boilerplate and prevents mistakes (forgetting the reducer). Chat is the most common state shape.

WHEN

Chatbots, tool agents, and any graph where messages plus a few extra fields are enough.

HOW

`class MyState(MessagesState): turn_count: int`

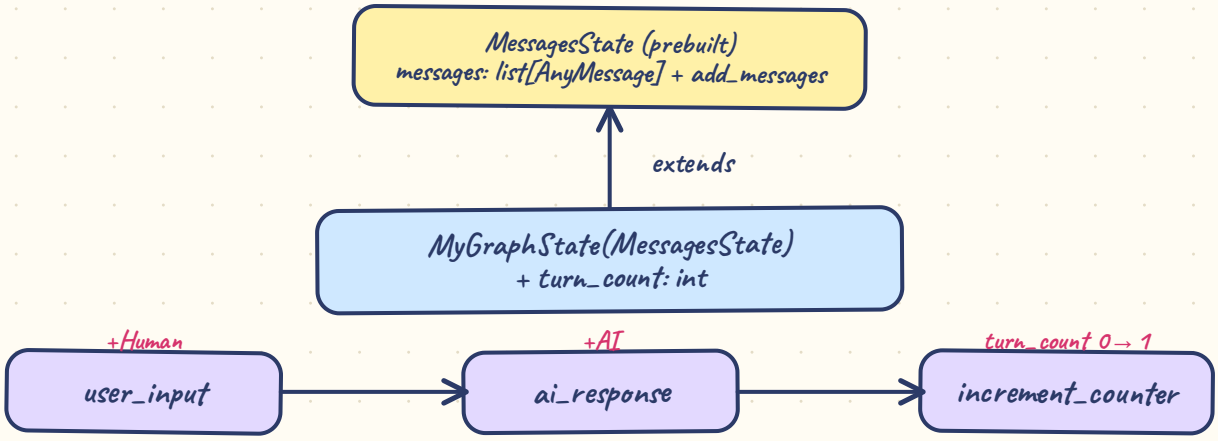


Fig 6.1 — Inherit the prebuilt, add your own keys, run three nodes

The demo graph: no LLM needed

06-graph-messages.py — trimmed

```python
class MyGraphState(MessagesState):
    turn_count: int                        # plain key ⇒ overwrite reducer

def user_node(state):                      # simulates the human
    return {"messages": HumanMessage(content="What's the weather like today?")}

def ai_node(state):                        # simulates the model
    last = state["messages"][-1]
    return {"messages": AIMessage(content=f"I've received: '{last.content}'")}

def counter_node(state):
    return {"turn_count": state["turn_count"] + 1}   # read, then write
# START → user_input → ai_response → increment_counter → END
```

```
messages: [HumanMessage("What's the weather…"), AIMessage("I've received…")]
turn_count: 1
```

How the pieces behave

Detail	What happens	Lesson
Initial input `{"turn_count": 0}`	`messages` is optional; it starts empty	State
`state["messages"][-1]`	Last message object. Use `.content` for text	Messages
`turn_count` *has no reducer*	Overwritten, so the node computes the new total itself	Reducers
Return a single message	Allowed. `add_messages` wraps it in a list	add_messages

Engineering insight

Fake nodes make great tests. A node that returns a canned `AIMessage` lets you test routing, state and reducers *without* network calls, cost or flakiness. LangChain also offers fake chat models (e.g. `GenericFakeChatModel`) for the same reason.

Decision rule

- Need ***only chat***? Use `MessagesState` directly.
- Need chat ***plus*** data (user id, summary, counters)? ***Subclass*** it.
- Need a different message shape or custom reducer? Write your own TypedDict with `add_messages`.

Watch out / Debug

Reading `state["turn_count"]` when the caller did not supply it fails, because the key is absent. Either require it in the input or use `state.get("turn_count", 0)`. Better: give it a reducer.

python — *improved pattern*

```python
# Safer: reducer returns deltas, so the node never reads the old value
class MyGraphState(MessagesState):
    turn_count: Annotated[int, operator.add]
def counter_node(state): return {"turn_count": 1}
```

2026 update

In 1.2.16 `MessagesState.__annotations__` shows `messages: Annotated[list[AnyMessage], add_messages]` (verified). Message classes are re-exported from `langchain.messages` in LangChain 1.x; `langchain_core.messages` still works.

Interview preparation

Q1. What is `MessagesState`? — Warm-up

A prebuilt TypedDict state with a `messages` list and the `add_messages` reducer. You subclass it to add fields.

Q2. How do you add extra state variables to it? — Warm-up

Subclass: `class MyState(MessagesState): turn_count: int`. The parent keys are inherited.

Q3. Why does the counter node read the old value and add 1, and how would you improve it?

Core

The key has no reducer, so it overwrites; the node must compute the total. Improvement: `Annotated[int, operator.add]` and return `1`. That removes the read, so parallel nodes cannot overwrite each other.

Q4. How do you test a graph that normally calls an LLM?

Core

Swap the LLM node for a deterministic fake that returns canned `AIMessage`s (or a fake chat model). Assert on final state and path taken. Keep a few real-model integration tests separately.

Q5. When would you not use `MessagesState`?

Deep dive

When messages are not the center of state (pure data pipelines), when you need a non-standard message key or custom merge logic, or when you want a strict Pydantic schema. Otherwise it is the default.

Q6. Design the state for a support bot that also tracks user id, ticket status and a running summary.

Deep dive

- `class SupportState(MessagesState)` with `user_id: str` (set once), `ticket_status: str` (overwrite), `summary: str` (overwrite, rewritten by a summary node).
- Put `user_id` in ***runtime context*** (Lesson 9) if it never changes during the run.
- Trim old messages after summarising; use a checkpointer keyed by `thread_id`.

Key takeaways

- `MessagesState` = `messages` + `add_messages`, prebuilt.
- ***Subclass*** it to add keys. Parent keys are inherited.
- Counters without reducers overwrite. Prefer reducers + deltas.
- Fake nodes = fast, free, deterministic tests.

LESSON 07

7 Nodes

The action takers: signatures, arguments, updates, per-node policies.

Code 07-nodes.py (no transcript provided; verified by running)

In one line: a node is **just a function** that reads state (and optionally config or runtime) and returns an update.

WHAT

The **action taker** of the graph: call an LLM, run code, validate, draft, send. Python or JavaScript functions.

WHY

Nodes are the only place state changes. Small, focused nodes make a graph readable, testable and retryable.

WHEN

Every step of work. Split a node when it does two jobs or needs a different retry policy.

HOW

`def node(state) → dict`, then `builder.add_node("name", node)`.

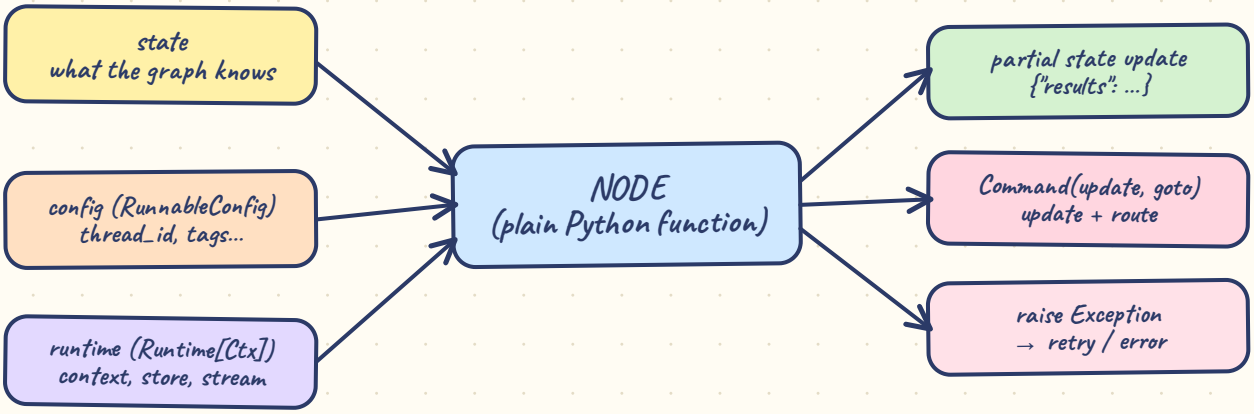


Fig 7.1 — What goes into a node, and what can come out

Three node signatures

Signature	Gives you	Use for
`(state)`	Current state	Most nodes
`(state, config: RunnableConfig)`	`config["configurable"]` such as `thread_id`, tags, callbacks	Per-run identifiers, tracing
`(state, runtime: Runtime[Ctx])`	`runtime.context` (typed run data), `store`, `stream_writer`	Dependencies, user info, long-term memory, custom streaming

07-nodes.py *trimmed*

```python
class ContextSchema(TypedDict):
    user_id: str

def plain_node(state: GraphState) -> dict:
    return {"results": f"Hello, {state['input']}!"}

def node_with_config(state, config: RunnableConfig) -> dict:
    thread_id = config.get("configurable", {}).get("thread_id")
    return {"results": "Config access successful."}

def node_with_runtime(state, runtime: Runtime[ContextSchema]) -> dict:
    user_id = runtime.context["user_id"]
    return {"results": "Runtime access successful."}

builder = StateGraph(GraphState, context_schema=ContextSchema)
app.invoke(input={"input": "World"},
           config={"configurable": {"thread_id": "user-1234"}},
           context={"user_id": "alice_smith"})
```

```
Executing plain_node...
→ In node, accessed 'thread_id' from config: user-1234
→ In node, accessed 'user_id' from runtime: alice_smith
{'input': 'World', 'results': 'Runtime access successful.'}
```

← Each node overwrote 'results' (no reducer), so only the last write survives.

Good node design

1. **One job** per node. "draft_email" and "send_email" are two nodes.
2. **Read** only the state keys you need. **Return** only keys you change.
3. **Side effects** (send, pay, write) go in their own node so you can approve, retry, or skip them.
4. **Raise on real errors.** Let retry policies or handlers act (Lesson 12). Do not swallow exceptions into state.

Per-node superpowers (add_node options)

Option	What it does
`retry_policy`	Retry on failure with backoff. Accepts a policy or a sequence of policies
`cache_policy`	Cache a node's result so identical input skips recomputation
`timeout`	Bound how long a node may run
`error_handler`	A node that handles the failure after retries
`input_schema`	Give this node a narrower view of state
`defer`, `metadata`, `destinations`	Delay until other branches finish; tag the node; declare routing targets for drawing

2026 update

These options are in the installed `StateGraph.add_node` signature (verified on 1.2.16): `defer, metadata, input_schema, retry_policy, cache_policy, error_handler, destinations, timeout, trace_policy`. Run-scoped dependencies belong in `runtime.context`, not `config["configurable"]`; keep `configurable` for things like `thread_id`. If you omit the name, `add_node(fn)` uses the function's name (verified).

Watch out / Debug

- The repo's `build_graph() → StateGraph` annotation is wrong: it returns a ***compiled*** graph.
- A typo in a ***returned key*** is silently dropped (see Lesson 11 for a real case). Unit-test node outputs against the schema.
- Do not mutate `state` in place. Return updates.

Interview preparation

Q1. What is a node and what does it return? Warm-up

A function that takes state and returns a ***partial update*** (a dict of changed keys), or a `Command` for update + routing.

Q2. Do node names have to match function names? Warm-up

No. The first argument of `add_node` is the name used by edges. If you omit it, the function name is used.

Q3. `config` vs `runtime.context`: when do you use which? Core

config: framework-level run settings (`thread_id`, tags, recursion limit). ***runtime.context***: your own typed, run-scoped data (user id, DB handle, model choice). Context is typed and not part of state, so it is not checkpointed.

Q4. A node calls an external API. How do you make it production-safe? Core

- Add a ***retry policy*** for transient errors only.
- Set a ***timeout***.
- Make the call ***idempotent*** (idempotency key) because retries can repeat it.
- Return a clear update or raise; do not hide failures.

Q5. A node returns a key that is not in the schema. What happens? Deep dive

In my test on 1.2.16 with TypedDict state the unknown key was ***silently ignored*** and the graph continued. That makes typos dangerous. Guard with tests that compare returned keys to the schema, or use a stricter schema.

Q6. How would you decide node granularity for a RAG answer pipeline? Deep dive

Split at boundaries where you need different ***retries, caching, observability, or approval***: rewrite_query → retrieve (cache + retry) → grade → generate → cite. Too coarse hides latency; too fine adds state plumbing and trace noise.

Key takeaways

- Node = function. Input ***state*** (+ optional ***config*** or ***runtime***). Output a ***partial update***.
- Use `runtime.context` for typed run data, `config` for thread and tracing.
- One job per node. Side effects in their own nodes.
- Per-node ***retry, cache, timeout, error handler*** are built in.

LESSON 08

8 Edges

Fixed and conditional edges: routing, joins, loops and failure modes.

Code 08-edges.py (no transcript provided; verified by running)

In one line: edges decide **what runs next**: fixed (A → B) or conditional (a function inspects state and picks).

WHAT

The connectors that define control flow. Normal edges are fixed. Conditional edges call a ***router function***.

WHY

Branching, loops and early exit are what make agents *agents*. Keeping routing in edges keeps nodes simple.

WHEN

Fixed edges for known order. Conditional for validation, retries, tool-or-answer, and multi-way routing.

HOW

`add_edge(a, b)` and `add_conditional_edges(a, router, path_map)`.

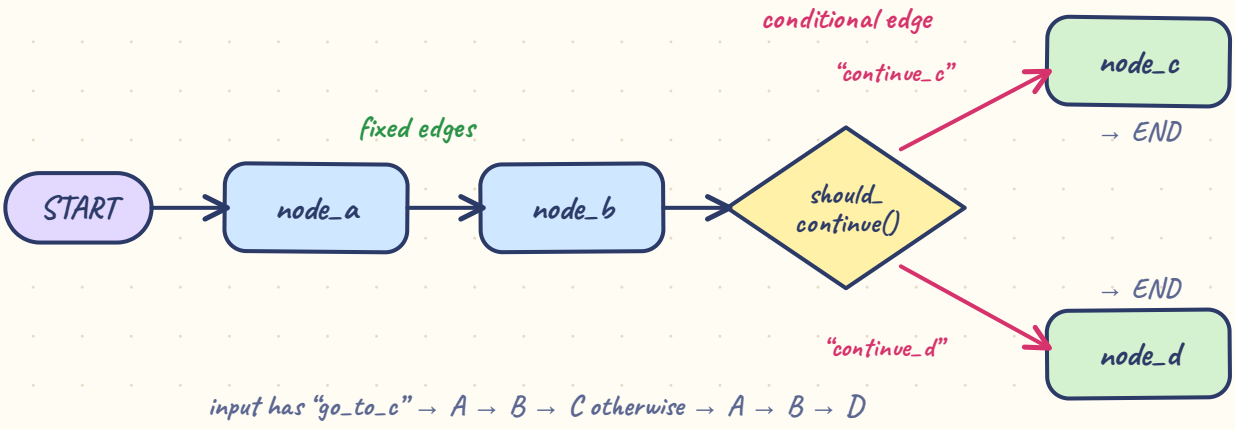


Fig 8.1 — Normal edges, then a conditional edge after node_b

08-edges.py — trimmed

```python
def should_continue(state: GraphState) -> str:
    if "go_to_c" in state["input"]:
        return "continue_c"        # must match a key in the map
    return "continue_d"

builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
builder.add_conditional_edges(
    "node_b", should_continue,
    {"continue_c": "node_c", "continue_d": "node_d"})
builder.add_edge("node_c", END)
builder.add_edge("node_d", END)
```

```
Executing Node A...  Node B...  Evaluating conditional edge...
→ Condition not met: Routing to Node D.
{'input': 'Hello…', 'execution_path': ['node_a', 'node_b', 'node_d']}
```

Edge toolbox

Tool	Use it for	Notes
`add_edge(a, b)`	Fixed order	Names, not functions
`add_edge([a, b], c)`	***Join***: run c after *all* of a and b	Fan-in after parallel work
`add_conditional_edges(src, fn, map)`	Branching on state	Router returns a key (or node name)
Router with `Literal[...]` ***return***	Typed routes, no map needed	Also helps diagrams
`add_conditional_edges(START, fn)`	Choose the ***entry*** node	Dynamic start
Return `END` ***from router***	Stop early	Import `END`

Engineering lens

Decision rule

- ***Router = pure function.*** Read state, return a label. No I/O, no mutation.
- ***Unit-test routers*** as plain functions with fake state. Every branch, including the default.
- ***Always have a fallback*** route (else-branch). Unknown labels are bugs.
- Use an ***LLM router*** only when rules cannot decide. Use structured output and a fallback.

terminal — *failure modes, verified*

```
# unmapped label (verified):   KeyError: 'nope'
# endless loop (verified):     GraphRecursionError: Recursion limit of 5 reached
#                               without hitting a stop condition.
app.invoke(state, {"recursion_limit": 25})   # set a ceiling on purpose
```

Watch out / Debug

- ***KeyError in routing*** → the router returned a label not in the map. Print what it returned.
- ***GraphRecursionError*** → a cycle with no exit. Add a counter in state and a stop route.
- Edges are wired by ***node name***. A misspelt name fails at compile time. Good.

Engineering insight

The repo builds `execution_path` by reading the old list and returning a new one. Cleaner: `execution_path: Annotated[list[str], operator.add]` and return `["node_a"]`. Same output, no read-modify-write.

Choose	When
Fixed edge	Next step is always the same
Conditional edge	Routing logic is separate from the node's work
`Command(goto=…)`	The node itself knows the next step and also updates state (Lesson 11)
`Send`	You need a ***dynamic number*** of parallel branches (Lesson 10)

Interview preparation

Q1. Normal vs conditional edge?

Warm-up

Normal: always go from A to B. Conditional: a function reads state and picks the next node(s) at run time.

Q2. Do conditional edges need to be LLM-driven?

Warm-up

No. Usually plain Python. You may use an LLM when the decision is fuzzy (intent classification), but rules are cheaper, faster and testable.

Q3. What does the third argument of `add_conditional_edges` do?

Core

It is the ***path map***: router label → node name. It lets the router return meaningful labels and lets LangGraph draw and validate the routes. With a `Literal` return type or a list of nodes it can be omitted.

Q4. How do you build a retry loop (validate → fix → validate) safely?

Core

Cycle: `validate → (ok → END | not ok → fix → validate)`. Keep an `attempts` counter in state, route to END or a human after N tries, and set a `recursion_limit`.

Q5. Compare conditional edges, `Command`, and `Send`.

Deep dive

- ***Conditional edge:*** routing separated from node logic; static graph structure visible.
- ***Command:*** node returns update + goto together; good when the decision comes from the work done.
- ***Send:*** map-reduce; number of branches known only at run time; each gets its own input.

Q6. Production traffic hits a `KeyError` inside LangGraph's routing. Diagnose.

Deep dive

The router returned a label missing from the path map (new enum value, unexpected LLM output, None). Fix: normalise router output, add a default route, type with `Literal`, test with property-based inputs, and log the label before returning.

Key takeaways

- Edges define ***flow***. Fixed for known order, conditional for decisions.
- Routers are ***pure functions*** that return a label. Map labels to nodes.
- Loops need a ***counter*** and a ***recursion limit***.
- Pick edge vs `Command` vs `Send` by *where the decision lives.*

LESSON 09

9 The Runtime Context

Typed per-run data for dependencies and settings, kept out of state.

Code `09-runtime-context.py, 07-nodes.py (no transcript provided; verified by running)`

In one line: runtime context is typed, read-only data you pass per run (user, DB, model choice) that does **not** live in state.

WHAT

A schema (often a `dataclass`) given to `StateGraph(..., context_schema=…)`. Values arrive via `invoke(context=…)`.

WHY

Keeps state clean. Dependencies and run settings do not belong in data that is checkpointed and replayed.

WHEN

User ids, tenant ids, DB connections or clients, API URLs, model/provider choice, feature flags.

HOW

Define schema → pass `context_schema` → read `runtime.context.field` in a node.

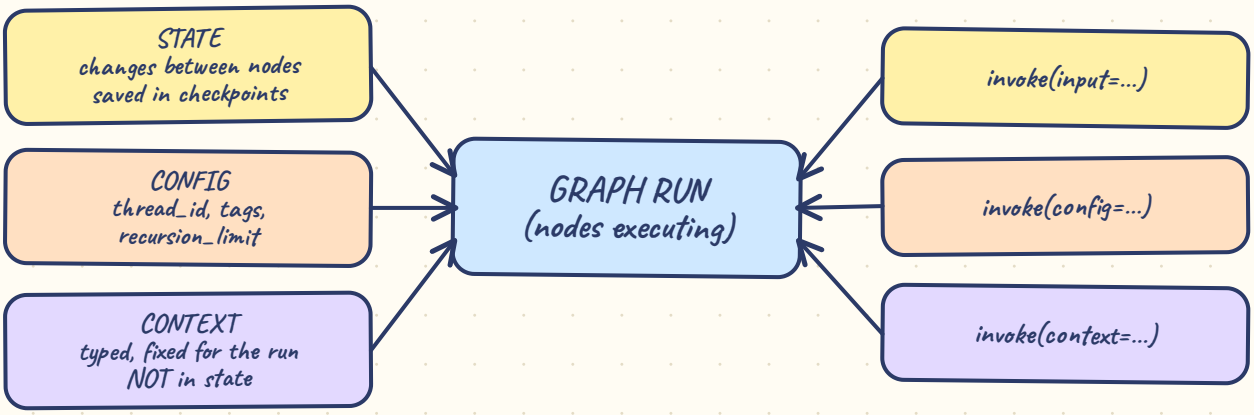


Fig 9.1 — Three ways data reaches a node

	State	Config	Context
Changes during run?	Yes (nodes update it)	No	No (fixed for the run)
Typed by you?	Yes (schema)	Loosely (dict)	**Yes** (context_schema)
Saved in checkpoints?	Yes	Partly (thread info)	**No**, so re-send each run
Typical content	Messages, results, counters	`thread_id`, tags	user_id, db, model name

09-runtime-context.py — trimmed

```python
@dataclass
class MyGraphContext:
    user_agent: str                     # required (no default)
    docs_url: str = "https://docs.langchain.com/"
    db_connection: str = "mysql://user:password@localhost:3306/my_db"

def context_access_node(state: GraphState, runtime: Runtime[MyGraphContext]):
    db = runtime.context.db_connection          # attribute access (dataclass)
    return {"result": f"Context accessed. DB: {db.split('//')[0]}..."}

builder = StateGraph(GraphState, context_schema=MyGraphContext)
graph.invoke(input=initial_state, context={"user_agent": "Default-Run"})
graph.invoke(input=initial_state, context={
    "user_agent": "Override-Run",
    "db_connection": "postgres://new_user@remote_host:5432/production"})
```

```
Run 1: DB mysql://user:password@localhost:3306/my_db  (default)
Run 2: DB postgres://new_user@remote_host:5432/production  (override)
```

← Defaults fill in what you do not pass. Required fields have no default.

Dict or dataclass context?

Context type	Access in node	Pros	Cons
`TypedDict` (Lesson 7)	`runtime.context["user_id"]`	Light	No defaults, no autocompletion
`@dataclass` (Lesson 9)	`runtime.context.user_id`	**Defaults**, dot access, IDE help	Slightly more code

Watch out / Debug

- **Missing required field** (verified): `TypeError: C.__init__() missing 1 required positional argument: 'user_agent'`.
- `context=None` (verified): the node fails with `AttributeError: 'NoneType'…`.
- **Secrets in defaults.** The demo hard-codes a DB string with a password. Never do that. Read secrets from env or a secret manager and inject them via context.

Decision rule

- Will a node **write** it? → **State**.
- Is it a **run identifier or tracing** value? → **Config**.
- Is it a **dependency or setting** that stays fixed? → **Context**.

Engineering insight

Testability: inject a fake DB or a cheaper model through context in tests, with no monkey-patching. **Multi-tenancy:** pass `tenant_id` in context so every node scopes queries correctly, and nothing tenant-specific is stored in shared state.

2026 update

`Runtime` is the single handle for run-scoped tools (verified fields: `context, store, stream_writer, previous, …`). This replaces the older habit of stuffing everything into `config["configurable"]`. Because context is not checkpointed, supply it again whenever you resume a thread.

Interview preparation

Q1. What is runtime context in LangGraph?

Warm-up

Typed, per-run data (like user id or a DB handle) declared with `context_schema` and passed with `invoke(context=…)`. Nodes read it through `runtime.context`.

Q2. How is context different from state?

Warm-up

State is mutable working data saved in checkpoints. Context is fixed for the run, typed separately, and not part of state or checkpoints.

Q3. How do dataclass defaults help here?

Core

They let callers override only what they need. The demo overrides just `db_connection`. Fields without defaults act as ***required*** inputs.

Q4. Where would you put the current user's id, and why?

Core

In ***context***, because it does not change during the run and nodes only read it. This keeps state small and avoids replaying identifiers from checkpoints.

Q5. You resume a paused thread (human approval) and a node crashes on `runtime.context`. Why?

Deep dive

Context is ***not persisted***. On resume you must pass `context=…` again, just like `config`. Wrap that in a helper so every entry point supplies it.

Q6. Design context for a multi-tenant SaaS agent.

Deep dive

- `@dataclass Ctx: tenant_id: str; user_id: str; model: str = "…"; db: DbClient`.
- Nodes build queries from `runtime.context.tenant_id`. State holds only conversation data.
- Secrets come from the environment. Tests inject a fake `db`.
- Audit logs include tenant and user from context.

Key takeaways

- Context = ***typed run-level dependencies***, separate from state.
- Declare `context_schema`, pass `context=` at invoke, read `runtime.context`.
- Dataclass defaults help, but ***never*** hard-code secrets. Context is ***not checkpointed***: re-supply on resume.

LESSON 10

10 The Send() API

Map-reduce in LangGraph: dynamic parallel workers and a join.

Code `10-send.py (no transcript provided; verified by running)`

In one line: `Send` lets a conditional edge launch **N parallel copies** of a node, each with its own input. N is decided at run time. This is map-reduce.

WHAT

A `Send(node_name, payload)` object returned from a conditional edge. Each one becomes a separate task.

WHY

Normal edges can only fan out to a ***fixed*** set of nodes. Real tasks have unknown size: pages, files, tickets, subtopics.

WHEN

Summarise many docs, research subtopics, classify a batch, call a tool per item, parallel agents.

HOW

Router returns `[Send("worker", {…}) for x in items]` + a ***reducer*** on the results key.

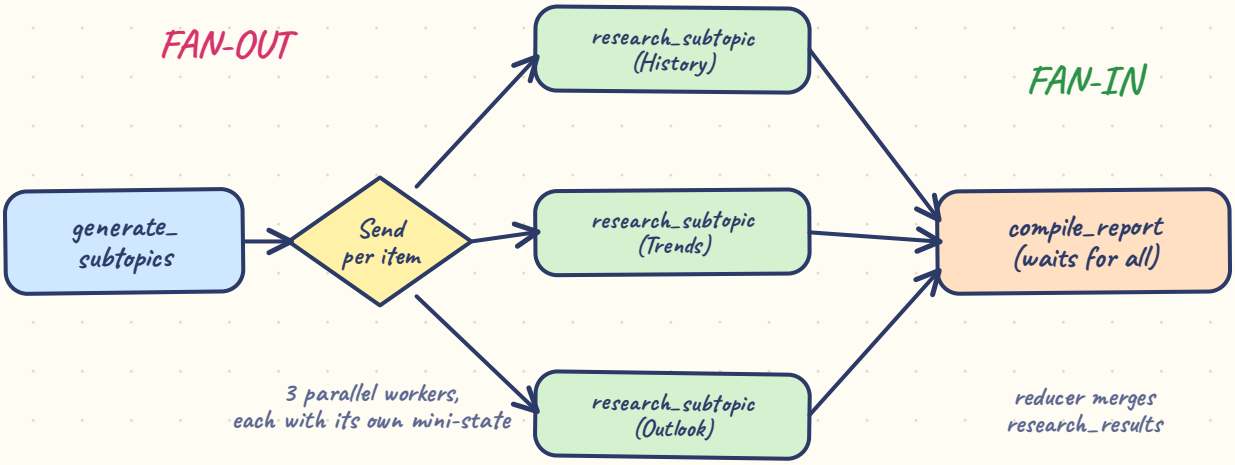


Fig 10.1 — Map (fan-out) → workers → Reduce (fan-in)

Pieces of the pattern

Piece	Role in 10-send.py
`OverallState`	`topic`, `subtopics`, `research_results` (with `add` reducer), `final_report`
`ResearchState`	***Private*** input for each worker: `{"subtopic": …}`
`generate_subtopics`	Planner: creates the list (dummy here, an LLM in real life)
`continue_to_research`	Router: returns one `Send` per subtopic
`research_subtopic`	Worker: returns `{"research_results": [result]}`
`compile_report`	Reducer step: reads all results, builds the report

10-send.py — trimmed

```python
def continue_to_research(state: OverallState):
    return [Send("research_subtopic", {"subtopic": s})
            for s in state["subtopics"]]

class OverallState(TypedDict):
    research_results: Annotated[List[str], add]   # merges parallel writes

builder.add_edge(START, "generate_subtopics")
builder.add_conditional_edges("generate_subtopics", continue_to_research)
builder.add_edge("research_subtopic", "compile_report")   # join
builder.add_edge("compile_report", END)
```

```
COMPREHENSIVE RESEARCH REPORT
1. Research findings on 'Artificial Intelligence - History'…
2. … 'Current Trends' …
3. … 'Future Outlook' …
```

Edge cases I tested

terminal — verified on langgraph 1.2.16

```
# 1) no reducer on the results key:
InvalidUpdateError: At key 'res': Can receive only one value per step.
# 2) Send list is EMPTY (no subtopics):
{'items': [], 'res': [], 'final': ''}     # compile_report never ran!
```

Watch out / Debug

- **No reducer → crash** when workers write the same key in one step.
- **Empty list → the reduce step is skipped**, because no worker finished to trigger it. Handle "no work" explicitly (conditional route straight to the final node).
- **Order is not guaranteed.** Put an id in each result, or sort in the reducer step.

Decision rule

- Number of branches known when you write the graph? → plain parallel **edges**.
- Unknown at build time and each branch needs its **own input**? → **Send**.
- Need to bound cost and rate limits? → set `max_concurrency` in config and cap list size.

Engineering insight

Trade-offs: faster wall-clock time, but token cost grows with N and a few slow workers delay the join. A failed worker fails the superstep; use **retry policies** (Lesson 12) on the worker node.

Interview preparation

Q1. What problem does `Send` solve? — Warm-up

Dynamic parallelism: launching a variable number of node instances, each with its own input, from one conditional edge.

Q2. Why does the worker use a different state from the overall graph?

Warm-up

Each worker only needs its slice (`subtopic`). A private input keeps workers independent. Their ***outputs*** are written to keys of the overall state and merged by that key's reducer.

Q3. You Send 500 items and hit provider rate limits. What do you do?

Deep dive

- Cap parallelism with `max_concurrency`, or batch items into chunks per worker.
- Add a ***retry policy*** with backoff + jitter on the worker.
- Make workers idempotent, and keep per-item ids so partial results can be re-run.

Q4. A run with zero items finishes but `final_report` is empty. Why and how do you fix it?

Deep dive

An empty `Send` list launches no worker, so the edge to `compile_report` never fires. Fix: make the router return a normal node name (e.g. `"compile_report"`) when the list is empty, or add a conditional edge that skips to the end.

Key takeaways

- ***Send*** = dynamic fan-out. One task per item, each with its own input.
- Always give the merged key a ***reducer***.
- Join happens when all workers finish. ***Empty list = join never runs.***
- Control cost: concurrency cap, retries, idempotent workers.

LESSON 11

11 Command (bonus lesson)

Update state and route in one step; resume and handoff patterns.

Code `11-command.py` (not in the timestamp list; no transcript; verified by running)

In one line: Command lets a node **update state and pick the next node** in a single return value.

Why Lesson 11 is here

This lesson is ***not in your timestamp list***, but the repo has `11-command.py`, and it sits between Send (10) and Retries (12). Included so the series is complete. No transcript was provided; it is explained from the code and verified by running it.

WHAT

A return type with `update`, `goto`, `resume`, and `graph` fields (verified).

WHY

When the decision comes ***from the work the node just did***, splitting it into node + router duplicates logic.

WHEN

Branching after an LLM/tool call, agent handoffs, and resuming after human input.

HOW

`return Command(update={…}, goto="next_node")`, with return type `Command[Literal[…]]`.

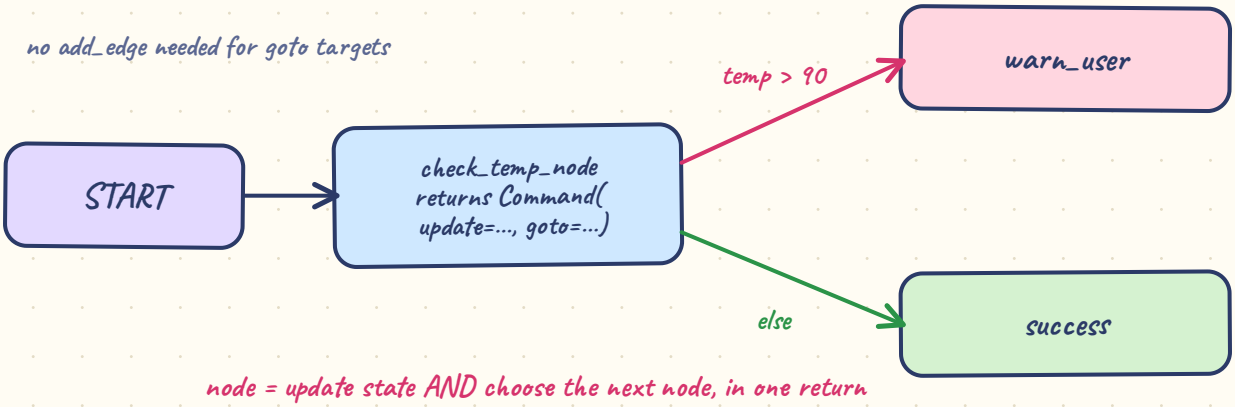


Fig 11.1 — One return = state update + routing

11-command.py *trimmed*

```python
def check_temp_node(state: GraphState) → Command[Literal["warn_user", "success"]]:
    temp = state["temperature"]
    if temp > 90:
        return Command(update={"status_message": "Routing to warning handler...",
                               "warning_sent": True},
                       goto="warn_user")
    return Command(update={"status_message": "Routing to Success Handler..."},
                   goto="success")

builder.add_node(check_temp_node)              # name = function name
builder.add_node(warn_user); builder.add_node(success)
builder.add_edge(START, "check_temp_node")     # goto targets need no edges
```

```
High: {'temperature': 100, 'warning_sent': True, 'final_action_performed': 'Warning Notification sent'}
Low : {'temperature': 40, 'final_action_performed': 'Temperate Safety Confirmed'}
```

Debug case (verified)

Real bug found by running it. The node writes `"status_message"`, but the state key is `status_messages` (plural). The final state has ***no status text at all*** and ***no error***. LangGraph silently ignored the unknown key. Fix the spelling, or type updates with a schema-checked helper.

python *typo that disappears*

```python
class GraphState(TypedDict):
    status_messages: str          # ← plural in the schema
...
Command(update={"status_message": "..."}, goto="warn_user")   # ← singular: DROPPED
# fix
Command(update={"status_messages": "..."}, goto="warn_user")
```

Command vs conditional edge

	Conditional edge	Command
Where is the decision?	A separate router function	Inside the node that did the work
Updates state?	No (the router only routes)	***Yes***, in the same return
Graph picture	Edges are visible	Edges hidden unless you add the `Literal` type or `destinations`
Best for	Clear, reusable routing rules	Decision depends on the node's result; handoffs

Other uses of Command

Field	Use
`update`	Partial state update (same shape as a normal return)
`goto`	Next node name, a list of names, or `Send` objects
`resume`	Resume a paused run after `interrupt()` (human approval). `invoke(Command(resume=…), config)`
`graph`	Target a ***parent*** graph from a subgraph (`Command.PARENT`): agent handoffs

2026 update

Human-in-the-loop in current LangGraph is built on `interrupt()` inside a node plus `Command(resume=…)` to continue; it needs a ***checkpointer*** and a `thread_id`. Nodes that only return `Command` can declare targets with `add_node(..., destinations=(…))` for drawing (verified in the signature).

Interview preparation

Q1. What does `Command` do? — Warm-up

A node returns `Command(update=…, goto=…)` to change state and choose the next node at once.

Q2. Do you need `add_edge` for nodes reached by `goto`? — Warm-up

No. `goto` routes at run time. For drawing and validation, annotate `Command[Literal[...]]` or use `destinations`.

Q3. When prefer `Command` over a conditional edge? — Core

When the routing depends on what the node just computed and you also need to write state. It avoids recomputing the decision in a separate router function.

Q4. The final state lacks `status_messages` but nothing failed. What is wrong? — Core

A key typo: the node updated `status_message`. Unknown keys are silently dropped. Fix the key; add a test that checks returned keys against the schema.

Q5. How does `Command` enable multi-agent handoffs? — Deep dive

A node inside a subgraph returns `Command(graph=Command.PARENT, goto="other_agent", update={…})` to move control to a sibling agent in the parent graph, passing state along.

Q6. Trade-offs of `Command` routing for maintainability? — Deep dive

Pros: logic in one place, fewer moving parts. Cons: flow is less visible in the diagram, harder to see all paths, and nodes now know graph topology (coupling). Mitigate with `Literal` types, `destinations`, and tests for each target.

Key takeaways

- ***Command*** = state `update` + `goto` in one return.
- Annotate `Command[Literal[...]]` so routes are known.
- ***Unknown update keys vanish silently.*** Guard against typos.
- Also powers human resume (`resume`) and handoffs (`graph=PARENT`).

LESSON 12

12 Retry Policies

Resilient nodes: backoff, jitter, what to retry, and what not to.

Code `12-retries.py (no transcript provided; verified by running)`

In one line: a **RetryPolicy** re-runs a failing node with **exponential backoff and jitter**, only for errors you choose.

WHAT

A policy object attached to **one node**: attempts, delays, backoff, jitter, and which exceptions to retry.

WHY

External calls fail briefly (timeouts, 5xx, rate limits). Retrying is cheaper than failing the whole run and losing progress.

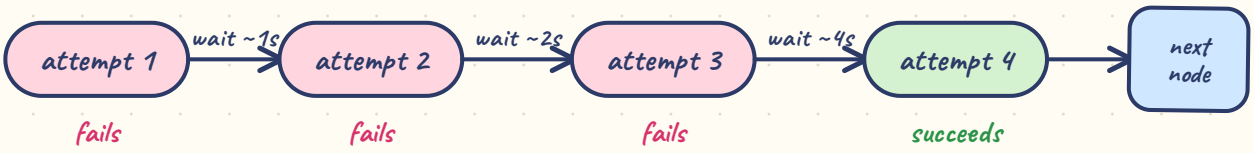
WHEN

Network/API nodes, LLM calls, flaky tools. **Not** for bugs or bad input.

HOW

```
builder.add_node("n", fn,
retry_policy=RetryPolicy(max_attempts=5, …))
```

interval = initial_interval × backoff_factor ^ (attempt-1), capped at max_interval, plus jitter



after max_attempts the last exception is raised → catch it or use an error handler

Fig 12.1 — Exponential backoff timeline (values from 12-retries.py)

The parameters

Parameter	Repo value	Default (verified)	Meaning
`max_attempts`	5	3	Total tries including the first
`initial_interval`	1.0 s	0.5 s	First wait
`backoff_factor`	2.0	2.0	Multiply the wait each time
`max_interval`	10 s	128 s	Ceiling on one wait
`jitter`	True	True	Random spread so clients do not retry in sync
`retry_on`	`APIError`	`default_retry_on`	Exception class, tuple, or function returning True/False

12-retries.py *trimmed*

```
builder.add_node(
    "fetch_weather", fetch_weather,
    retry_policy=RetryPolicy(
        max_attempts=5, initial_interval=1.0, backoff_factor=2.0,
        max_interval=10.0, jitter=True, retry_on=APIError))

# fetch_weather: 70% chance to raise APIError
```

```
❌ API call failed for San Francisco
✅ Successfully fetched weather for San Francisco
✨ Final Result: {'city': 'San Francisco', 'temperature': 18.8, 'conditions': 'Sunny'}
```

Do the math

- Waits for 5 attempts: **1, 2, 4, 8** s (4 gaps), each ±jitter. All under the 10 s cap.
- With a 70% failure rate, all 5 fail with probability $0.7^5 \approx 16.8\%$. So ~83% of runs succeed. That is why the demo wraps the call in `try/except`.
- Retries are a ***probability tool***, not a guarantee. Size attempts to the failure rate and the cost of waiting.

Engineering insight

What the default policy retries (verified in source): connection errors and HTTP ***5xx***. It does ***not*** retry `ValueError, TypeError, KeyError/LookupError, RuntimeError, OSError` and similar. Those usually mean a bug or bad input, where retrying only repeats the failure. Your own exceptions are retried by default unless you narrow `retry_on`.

Decision rule

- ***Retry*** transient failures: timeouts, 429, 5xx, connection resets.
- ***Do not retry*** validation errors, 4xx auth/permission, logic bugs, or non-idempotent side effects.
- Make the node ***idempotent*** (idempotency key) if the retry could repeat a payment or email.
- Set ***jitter on*** and respect ***Retry-After*** / rate limits.

Watch out / Debug

- ***Retry storms:*** many clients retrying together overload a recovering service. Backoff + jitter + attempt cap prevents this.
- ***Hidden latency:*** 5 attempts with backoff can add ~15 s. Fit it to your SLA.
- ***Swallowed errors:*** if `retry_on` catches everything, real bugs get retried three times, then fail the same way.

2026 update

- In the installed signature, `retry_policy` accepts a ***sequence*** of policies (different rules per error type).
- `add_node` also has `timeout` (`TimeoutPolicy` with `run_timeout` / `idle_timeout`) and an `error_handler` node. The docs note timeouts use cooperative asyncio cancellation, so they are reliable for ***async*** nodes and not for blocking sync code.
- Combine: ***timeout → retry → error handler / fallback node*** for a resilient external call.

Interview preparation

Q1. What does a `RetryPolicy` do? — Warm-up

Re-runs a failed node automatically with configurable attempts, backoff, jitter, and exception filter.

Q2. Why use exponential backoff and jitter? — Warm-up

Backoff gives a struggling service time to recover. Jitter spreads retries in time so many clients do not hit the service at the same instant.

Q3. Compute the waits for `max_attempts=5, initial_interval=1, backoff_factor=2, max_interval=10`. — Core

Four waits between five attempts: ***1, 2, 4, 8 s*** (plus jitter). A fifth wait would be 16 s but is capped at 10 s. Total ≈ 15 s of waiting in the worst case.

Q4. Which errors should you retry and which not? — Core

Retry transient ones (timeouts, 429, 5xx, connection errors). Never retry bad input (ValueError), auth/permission (401/403), or logic bugs. The default `retry_on` encodes this split.

Q5. A node sends an email, and retries cause duplicates. Fix it. — Deep dive

- Make the send ***idempotent***: pass an idempotency key derived from the run + step.
- Split the work: `prepare` (retryable) and `send` (guarded, maybe human-approved).
- Record "sent" in state or an external ledger, and skip if already sent.

Q6. Retries are not enough: the API stays down for minutes. What is your resilience design? — Deep dive

Layer it: ***timeout*** per attempt → ***retry*** with capped backoff → ***fallback*** (cached data, secondary provider, degraded answer) via an error handler or conditional edge → ***circuit breaker***/alerting outside the graph. Checkpointing lets a failed run ***resume*** later from the failed step instead of restarting.

Key takeaways

- `retry_policy` is set ***per node***: attempts, backoff, jitter, `retry_on`.
- Retry ***transient*** failures only. Make retried nodes ***idempotent***.
- Backoff 1, 2, 4, 8 s; with 70% failure, 5 attempts still fail ~17% of the time.
- Build layers: timeout → retry → fallback → alert.

APPENDIX A

Series Concept Map

How all twelve lessons connect.

Read this page last. If you can explain every box and arrow, you have the whole course.

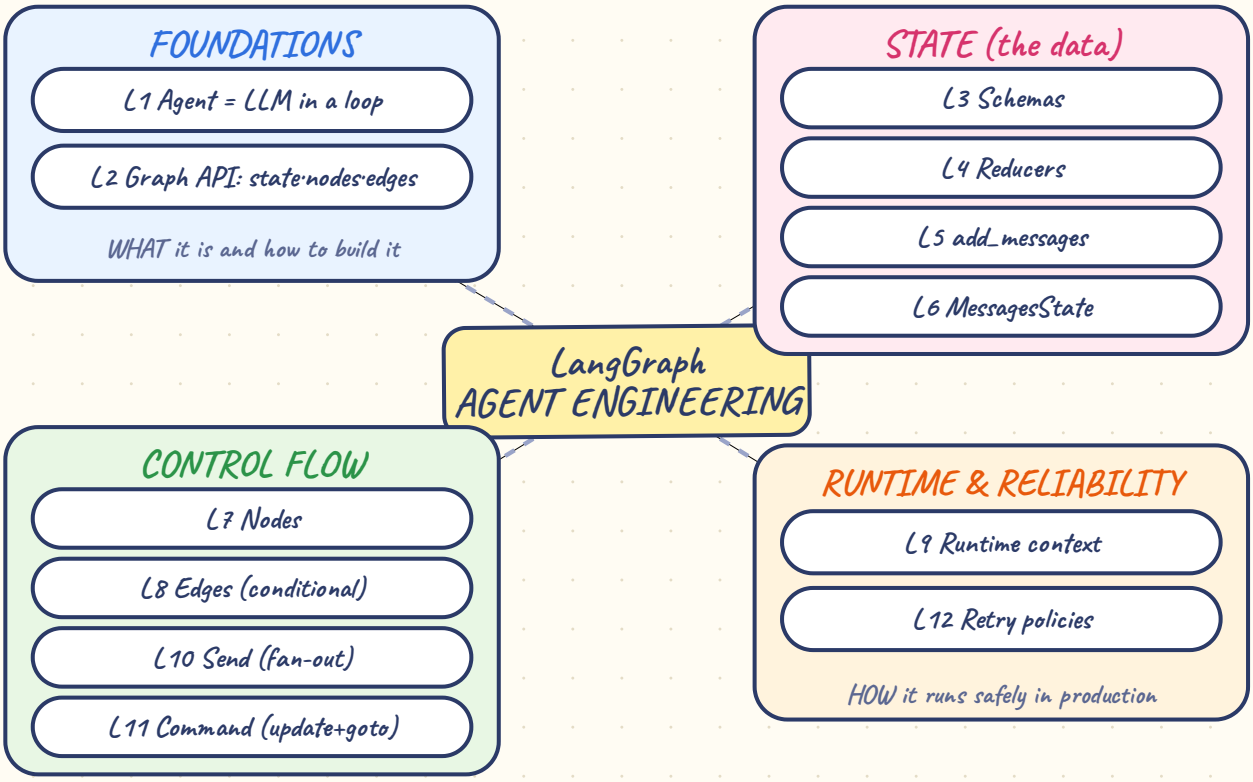


Fig A.1 — The whole course on one page

How the lessons build on each other

Layer	Question it answers	Lessons
Mental model	What is an agent and what is a graph?	1, 2
Data	What does the agent remember and how does it change?	3, 4, 5, 6
Behaviour	What does each step do, and what runs next?	7, 8, 10, 11
Environment	What does a run depend on, and what if it fails?	9, 12

Engineering insight

One sentence per lesson, for quick recall: 1 agent = LLM loop · 2 state+nodes+edges · 3 schema choices · 4 reducers merge writes · 5 add_messages for chat · 6 MessagesState subclass · 7 nodes are functions · 8 edges route · 9 context = typed run deps · 10 Send = dynamic parallel · 11 Command = update+goto · 12 RetryPolicy = backoff for transient errors.

APPENDIX B

B Workflow & Agent Patterns

From the repo's workflows.md (LangGraph docs): six building blocks and how they map to the Graph API.

Idea: real systems are **compositions** of these patterns. Start simple and add one pattern at a time.

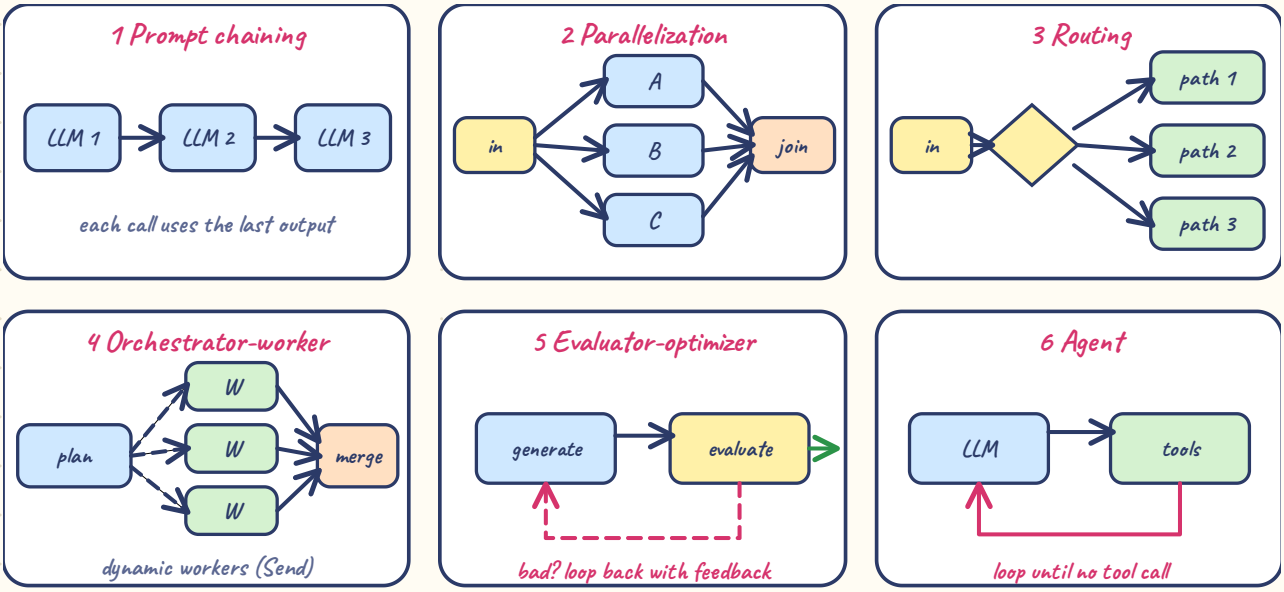


Fig B.1 — The six patterns

Pattern	Use when	LangGraph building blocks	Lesson
Prompt chaining	Task splits into fixed steps; each uses the last output	Sequential `add_edge`; one node per step	2
Parallelization	Independent sub-tasks, or vote on the same task	Several edges from one node; join into one node (fan-in)	8, 4
Routing	Classify input, then send to a specialist	`add_conditional_edges` with a router	8
Orchestrator-worker	Sub-tasks unknown until run time	`Send` + reducer + synthesiser node	10
Evaluator-optimizer	Quality can be judged; iterate until good	Conditional edge that loops back; attempt counter	8
Agent	Open-ended; path cannot be predefined	LLM node + tool node + loop on "has tool calls?"	1, 5, 7

Decision rule

Escalate in this order: ***single LLM call → chain → route → parallelise → orchestrate → agent.*** Each step adds cost, latency and debugging effort. Stop at the first one that meets quality. (Model names inside `workflows.md` are older examples; use current ones.)

APPENDIX C

Debugging Playbook

Symptom → likely cause → fix. Every row marked ✓ was reproduced while preparing these notes.

Symptom	Likely cause	Fix	C
List has only the last item	No reducer; default ***overwrites*** ✓	`Annotated[list, operator.add]`	4
Counter stays at 1	Overwrite again ✓	Reducer `current + new`; return deltas	3, 4
`InvalidUpdateError … only one value per step`	Parallel nodes wrote the same key without a reducer ✓	Add a reducer, or write different keys	4, 10
Key never appears in final state	Typo in returned key; unknown keys are ***silently dropped*** ✓	Fix spelling; test returned keys vs schema	7, 11
`ImportError: Install grandalf`	ASCII drawing needs an extra package ✓	`pip install grandalf`	2
`KeyError: 'label'` ***at a conditional edge***	Router returned a label not in the path map ✓	Add map entry/default; type with `Literal`	8
`GraphRecursionError`	Cycle without exit ✓	Counter in state; exit route; set `recursion_limit`	8
`TypeError … missing … argument` ***at invoke***	Required context field not passed ✓	Pass `context={…}` with all required fields	9
`AttributeError: 'NoneType'` ***on context***	`context=None` ✓	Pass a dict; default in schema	9
Final step never runs after `Send`	Empty Send list ✓	Route straight to the join when no work	10
Bot forgets earlier turns	No checkpointer or no `thread_id` ✓	`compile(checkpointer=…)` + `thread_id`	5
400: tool call without tool message	History trimmed between AI tool_calls and ToolMessage	Trim as pairs; always return a ToolMessage	5
Retries repeat a bug	`retry_on` too broad	Retry only transient exceptions	12
Duplicate side effects	Retried non-idempotent node	Idempotency key; split prepare/send	12

A 6-step method that always works

1. **Reproduce** with the smallest input and a fake LLM node.
2. **Print the state** after each node (use `stream_mode="values"` or `"updates"`).
3. **Check the keys** each node returns against the schema.
4. **Check reducers** for every key written more than once.
5. **Check routing**: print the router's return value.
6. **Add a test** for the bug so it cannot return.

APPENDIX D

Source Audit & 2026 Update Notes

What matched, what differed, and what changed in the ecosystem.

Transcript ↔ code: differences

Where	Transcript says	Code / reality	Impact
Lesson 4 data	animals = lion, **buffalo**	`lion, tiger`	None (demo values)
Lesson 5 AI-message merge	Consecutive AI messages are merged	Appended; only **same id** replaces ✓	Do not rely on merging
Lesson 5 tool messages	"LLM sends a tool message"	LLM sends `AIMessage(tool_calls)`; tool result is `ToolMessage`	Conceptual accuracy
Lesson 2 install	`pip install langgraph` includes everything	Drawing needs `grandalf`; model SDKs need their own package	Setup
Lessons 6–12	No transcript supplied	Notes written from code; behaviour verified by running	—
Lesson 11	Not in your timestamp list	Code exists; included as a bonus lesson	—

Issues in the repo code

File	Issue	Suggested fix
11-command.py	`status_message` vs `status_messages` ✓ silently dropped; also "'C" printed for a plain number	Match key names; test returned keys
07-nodes.py	`build_graph() → StateGraph` but returns compiled graph	Annotate as `CompiledStateGraph` or drop
09-runtime-context.py	Default DB string contains a password	Read secrets from env; no secrets in defaults
08-edges.py	Read-modify-write on `execution_path`	Use `operator.add` reducer
03 / 04	Most demos are commented out; one runs at a time	Wrap each in a function or `if __name__` switch
02-graph-api.py	Ends with `draw_ascii()`, crashes without grandalf ✓	Install it or guard with try/except
12-retries.py	17% of runs end in the exception branch (by design)	Seed `random` for reproducible demos
workflows.md	Older model names and package notes	Refresh to current model ids

What changed by 2026 (checked where marked ✓)

Topic	Then (2024–25)	Now
Version	LangGraph 0.x, frequent API shifts	**1.x stable** (1.0 in Oct 2025; tested on 1.2.16 ✓)
Prebuilt agent	`langgraph.prebuilt.create_react_agent`	`langchain.agents.create_agent` ✓ (with middleware, context_schema, checkpointer)
Run-scoped data	Stuffed into `config["configurable"]`	`context_schema` + `Runtime` ✓
I/O schemas	`input=` , `output=` args	`input_schema=` , `output_schema=` ✓
Checkpointer name	`MemorySaver`	`InMemorySaver` ✓ (dev); database savers for prod
Node controls	Retry only	Retry, cache, timeout, error handler ✓
Human approval	Static breakpoints	`interrupt()` + `Command(resume=…)`
Messages import	`langchain_core.messages`	Also `langchain.messages` ✓

← Always re-check against the current docs: this ecosystem moves quickly.

APPENDIX E

One-Page Cheat Sheet

Everything you need to rebuild a graph from memory.

cheat-sheet.py — skeleton (shape only)

```python
from typing import TypedDict, Annotated
from operator import add
from dataclasses import dataclass
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.graph.message import add_messages
from langgraph.runtime import Runtime
from langgraph.types import Command, Send, RetryPolicy

class State(TypedDict):                          # 1. STATE (+ reducers)
    messages: Annotated[list, add_messages]      #    chat history
    steps:    Annotated[list[str], add]          #    append
    answer:   str                                #    overwrite

@dataclass
class Ctx:                                       # run-scoped, typed, not in state
    user_id: str

def work(state: State, runtime: Runtime[Ctx]) -> dict:   # 2. NODE
    return {"steps": ["work"], "answer": f"hi {runtime.context.user_id}"}

def route(state: State) -> str:                  # 3. ROUTER (pure)
    return "end" if state["answer"] else "again"

g = StateGraph(State, context_schema=Ctx)
g.add_node("work", work, retry_policy=RetryPolicy(max_attempts=3))
g.add_edge(START, "work")
g.add_conditional_edges("work", route, {"end": END, "again": "work"})
app = g.compile()                                # 4. COMPILE (+ checkpointer)
app.invoke({"messages": []}, {"recursion_limit": 25}, context=Ctx("ada"))
```

Need	Use
Overwrite	plain type
Append list	`Annotated[list, add]`
Chat history	`add_messages`
Counter	`(cur, new) → cur+new`
Hide internals	`input_schema / output_schema`

Need	Use
Branch	`add_conditional_edges`
Update + route	`Command`
N parallel	`Send` + reducer
Flaky API	`retry_policy`
Memory	`checkpointer + thread_id`

Seven golden rules

1. State first, then nodes, then edges, then compile.
2. Every key written twice needs a reducer.
3. Return deltas, not totals.
4. Routers are pure; test each branch.
5. Every loop has an exit and a recursion limit.
6. Retry transient errors only; make retried nodes idempotent.
7. Secrets never in state or defaults.

APPENDIX F

Mock Interview Round

Rapid-fire recall + one system-design question.

Rapid-fire (answer in one breath)

Question	Strong short answer
1. What are the three parts of a LangGraph agent?	Prompt, LLM, tools, run in a loop.
2. Why compile a graph?	Validates structure and returns an executable runnable with invoke/batch/stream.
3. Default reducer?	Overwrite.
4. Reducer signature?	`(current, new) → merged`.
5. Which reducer for chat history?	`add_messages`.
6. Why is the LLM stateless and what follows?	Each call needs full history; keep it in state or a checkpointer.
7. How to subclass chat state?	`class S(MessagesState): extra: int`.
8. config vs context?	Config = thread/tracing/limits. Context = typed run dependencies.
9. Conditional edge returns an unknown label?	`KeyError`; add a default route.
10. How to stop infinite loops?	Counter in state + `recursion_limit`.
11. When Send?	Dynamic number of parallel branches, each with its own input.
12. Send with empty list?	No workers, so the join never fires.
13. Command vs conditional edge?	Command = update + route in the node; edge = separate pure router.
14. Which errors to retry?	Transient (timeout, 429, 5xx). Not bugs or bad input.
15. Backoff for 5 attempts at 1 s ×2?	1, 2, 4, 8 s (+ jitter).
16. Make a retried send-email safe?	Idempotency key; split prepare/send.

System design

Q1. Design a customer-support agent that answers from a knowledge base, can issue refunds under $50, escalates otherwise, and never double-refunds. Deep dive

State: `MessagesState` + `ticket_id`, `intent`, `refund_amount`, `attempts`.

Context: `tenant_id`, `user_id`, DB/API clients. Secrets from env.

Graph: classify (router) → *answer* (RAG: retrieve with cache+retry → generate) | *refund* | *escalate*.

- **Refund path:** validate amount ≤ $50 in **code** (not the LLM) → `interrupt()` for approval if close to limit → `issue_refund` node with **idempotency key** (ticket + amount).
- **Resilience:** timeout + retry (transient only) on API nodes; fallback to "escalate to human".
- **Safety:** loop counter + recursion limit; log every tool call; trim history; checkpointer by `thread_id`.
- **Testing:** fake LLM nodes for routing tests; replay failing threads from checkpoints.

Tip: name the state, control flow, failure modes and trade-offs in every design answer.